

EE-2003

Computer Organization and Assembly Language

Week 13 – Lecture 26

Department of Computer Science/Software Engineering/ Artificial Intelligence
FAST-NUCES Peshawar

Outline

- 1 Quick Review
- 2 Number Printing in Assembly & its Algorithm
- 3 The DIV Instruction (Integer Division)

Quick Review: What was in the Last Lecture?

- Display Memory
- Declaring Characters and Strings in Assembly
- Clearing and Printing on the Screen

Number Printing in Assembly

- In assembly, numbers are stored in **binary form**, not as text.
- For example:
 - The number 10 is stored as binary 00001010, not as the ASCII characters '1' and '0'.
- If a binary number is written directly to the screen, it shows as a meaningless symbol.
- To display it correctly, we must convert it to its **ASCII representation**.

Number Printing Algorithm

Goal: Convert a binary number to readable digits (e.g., $253 \rightarrow '2' '5' '3'$).

- 1 Divide the number by **10** (for decimal base).
- 2 The **remainder** gives the rightmost digit.
- 3 Convert the remainder to ASCII by adding $0x30$.
- 4 Push this digit onto the stack.
- 5 Repeat the process until the quotient becomes 0.
- 6 Pop digits from the stack and print them left to right.

Using the stack ensures digits appear in the correct order.

Why Use the Stack?

- Each division gives the **last digit first**.
- Example: For 253, remainders appear as 3, 5, 2.
- If printed directly, it shows **352 (incorrect)**.
- By pushing each digit to the stack:
 - Push order: 3, 5, 2
 - Pop order: 2, 5, 3 (correct)
- The **stack (LIFO)** reverses the order automatically.

The DIV Instruction (Integer Division)

- Performs **unsigned integer division**.
- Two common forms:
 - DIV BL → divides 16-bit AX by 8-bit BL.
 - DIV BX → divides 32-bit DX:AX by 16-bit BX.
- Results are stored as:
 - Quotient → AL (8-bit) or AX (16-bit)
 - Remainder → AH (8-bit) or DX (16-bit)
- If the quotient is too large to fit, a **Divide Overflow Interrupt (Type 0)** occurs.

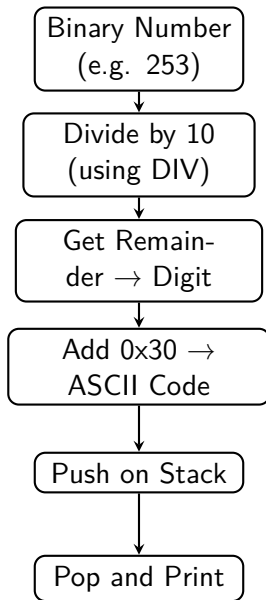
Step-by-Step Example: Printing 253

- 1 Number = 253, Base = 10
- 2 Divide 253 by 10 $\rightarrow$ Quotient = 25, Remainder = 3
- 3 Push ASCII(3) on stack
- 4 Divide 25 by 10 $\rightarrow$ Quotient = 2, Remainder = 5
- 5 Push ASCII(5)
- 6 Divide 2 by 10 $\rightarrow$ Quotient = 0, Remainder = 2
- 7 Push ASCII(2)
- 8 Pop and print $\rightarrow$ 2, 5, 3
*Final output on screen $\rightarrow$ **253***

Stack Order

2 (Top)
5
3 (Bottom)

Visual Diagram: Number to Display



Key Takeaways

- Numbers must be converted from binary to ASCII before printing.
- Adding 0x30 converts a digit (0–9) to its ASCII code.
- The **stack** ensures digits appear in correct order.
- The DIV instruction splits:
 - Quotient → next number to divide.
 - Remainder → next digit to print.
- Overflow occurs if the quotient exceeds its register size.

Number Printing Algorithm: Overview

- Convert a binary number to ASCII for display.
- Steps:
 - ① Divide the number by base (e.g., 10 for decimal).
 - ② Remainder = rightmost digit.
 - ③ Convert remainder to ASCII (add 0x30 for decimal).
 - ④ Push digit on stack.
 - ⑤ Repeat until quotient = 0.
 - ⑥ Pop digits from stack to print in correct order.
- Stack ensures **last digit pushed** → **first printed**.

Number Printing: Subroutine Code Part 1

```
printnum: push bp
mov bp, sp
push es
push ax
push bx
push cx
push dx
push di
mov ax, 0xb800
mov es, ax           ; point ES to video memory
mov ax, [bp+4]       ; number to print
mov bx, 10           ; base 10
mov cx, 0            ; digit count
```

Number Printing: Subroutine Code Part 2

```
nextdigit:
    mov dx, 0           ; prepare 32-bit dividend
    div bx              ; divide AX by BX
    add dl, 0x30        ; convert remainder to ASCII
    push dx             ; save ASCII digit on stack
    inc cx              ; increment count
    cmp ax, 0
    jnz nextdigit       ; repeat if quotient not zero

mov di, 0               ; top-left of screen
nextpos:
    pop dx
    mov dh, 0x07        ; attribute byte (white on black)
    mov [es:di], dx
    add di, 2
    loop nextpos
```

Number Printing: Subroutine Cleanup and Main Program

```
pop di
pop dx
pop cx
pop bx
pop ax
pop es
pop bp
ret 2                ; remove parameter
```

```
start:
    call clrscr
    mov ax, 4529
    push ax
    call printnum
    mov ax, 0x4c00
    int 0x21
```

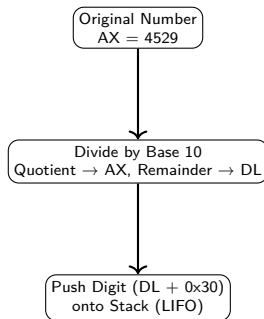
Key Points of the Algorithm

- Registers are saved/restored to preserve program state.
- ES → video memory base, DI → cursor position.
- BX → base (decimal, binary, octal, etc.)
- CX → number of digits pushed on stack.
- DX → holds remainder after division (DL for digit).
- Stack stores digits in reverse, popped in correct order.
- Algorithm is versatile: changing BX prints numbers in different bases:
 - BX=2 → binary
 - BX=8 → octal
 - BX=16 → hexadecimal (requires adjustment for A-F)

Output Example and Explanation

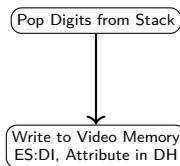
- For $AX = 4529$:
 - Stack after first loop: 0x34, 0x35, 0x32, 0x39
 - Popped and printed left to right $\rightarrow$ 4529 displayed at top-left.
- Each digit is converted to ASCII and printed with attribute byte.
- Cursor moves 2 bytes per character (word per screen cell).
- Changing BX prints the number in any desired base.

Number Printing: Division & Stack (Part 1)



- AX holds the quotient after each division.
- DL stores the remainder converted to ASCII.
- Stack ensures digits are reversed for proper order.

Number Printing: Pop & Display (Part 2)



- Digits are printed left-to-right on screen.
- DI moves 2 bytes per character (ASCII + Attribute).
- ES points to video memory base 0xB800.
- CX keeps track of the number of digits to print.

Thank You!

Any Questions?

Final Thoughts:

- Strings and ASCII codes bridge software and hardware concepts.
- Direct video memory access (0xB8000) shows how bytes become visible text.
- Subroutines and stack management provide modularity, reliability, and order control.
- Number printing demonstrates the use of stack to reverse digits for correct display.
- “Hello World” and number printing examples highlight the power of low-level programming.

“In assembly, every visible output — text or number — is a precise sequence of bytes, where logic directly meets hardware.”